

# Solutions to the SageMath Worksheet

August 2026

The computations below were checked with SageMath 10.8. We begin every session by creating the four bases that occur in the worksheet.

```
Sym = SymmetricFunctions(QQ)
m = Sym.monomial()
e = Sym.elementary()
p = Sym.powersum()
s = Sym.schur()
```

Calling, for example, `e(X)` converts the symmetric function `X` to the elementary basis.

1. **Solution.** The following code computes all the requested matrices.

```
for n in range(1, 7):
    order = list(Partitions(n))
    A = m.transition_matrix(s, n)
    upper = all(A[i,j] == 0
                for i in range(A.nrows())
                for j in range(i))
    print("n =", n, "order =", order)
    print(A)
    print("upper triangular:", upper)
    print("determinant:", A.det())
```

The method `m.transition_matrix(s,n)` writes the degree- $n$  monomial basis in the Schur basis. Sage orders partitions in decreasing lexicographic order. The matrices are

```
n=1: [1]

n=2: [1 -1]      n=3: [1 -1 1]
     [0  1]          [0  1 -2]
                       [0  0  1]

n=4:
```

```

[1 -1 0 1 -1]
[0 1 -1 -1 2]
[0 0 1 -1 1]
[0 0 0 1 -3]
[0 0 0 0 1]

```

n=5:

```

[1 -1 0 1 0 -1 1]
[0 1 -1 -1 1 1 -2]
[0 0 1 -1 -1 2 -2]
[0 0 0 1 -1 -1 3]
[0 0 0 0 1 -2 3]
[0 0 0 0 0 1 -4]
[0 0 0 0 0 0 1]

```

n=6:

```

[1 -1 0 1 0 0 -1 0 0 1 -1]
[0 1 -1 -1 0 1 1 0 -1 -1 2]
[0 0 1 -1 -1 0 1 -1 1 -2 2]
[0 0 0 1 0 -1 -1 1 1 1 -3]
[0 0 0 0 1 -1 1 1 0 -1 1]
[0 0 0 0 0 1 -2 -2 0 4 -6]
[0 0 0 0 0 0 1 0 -1 -1 4]
[0 0 0 0 0 0 0 1 -1 1 -1]
[0 0 0 0 0 0 0 0 1 -3 6]
[0 0 0 0 0 0 0 0 0 1 -5]
[0 0 0 0 0 0 0 0 0 0 1]

```

Every matrix is upper triangular with diagonal entries 1, so every determinant is 1. This is the computational form of the unitriangularity of the Kostka matrix.

2. **Solution.** Here are examples with fewer than four parts, exactly four parts, and more than four parts.

```

examples = [[2], [2,1], [1,1,1,1], [1,1,1,1,1]]
for la in examples:
    print(la, s[la].expand(4))
    print("monomial form:", m(s[la]))

```

Sage uses variables  $x_0, x_1, x_2, x_3$ . The mathematical results are

$$s_2 = m_2 + m_{1,1}, \quad s_{2,1} = m_{2,1} + 2m_{1,1,1}, \quad s_{1^4} = m_{1^4}.$$

The abstract function  $s_{1^5} = m_{1^5}$  is nonzero. In contrast, the command

```
s[1,1,1,1,1].expand(4)
```

returns 0, because a Schur polynomial whose shape has more than four rows vanishes in four variables.

Thus `expand(4)` realizes an abstract symmetric function as a symmetric polynomial in four variables. The monomial coefficients are Kostka numbers: they are nonnegative integers counting semistandard tableaux.

3. **Solution.** The following loop performs every requested calculation.

```
families = [
    ("paths",    range(1,8), graphs.PathGraph),
    ("cycles",   range(3,8), graphs.CycleGraph),
    ("complete", range(1,8), graphs.CompleteGraph)
]
bases = [("m",m), ("e",e), ("p",p), ("s",s)]

for name, sizes, constructor in families:
    print("\n", name)
    for n in sizes:
        G = constructor(n)
        X = G.chromatic_symmetric_function(QQ)
        print("n =", n)
        for basis_name, B in bases:
            print(basis_name, B(X))
```

`chromatic_symmetric_function()` first returns the power-sum expansion, using

$$X_G = \sum_{A \subseteq E(G)} (-1)^{|A|} p_{\lambda(A)}.$$

The numbers of nonzero terms in the four bases are:

| family and sizes       | basis     | numbers of terms        |
|------------------------|-----------|-------------------------|
| $P_n, 1 \leq n \leq 7$ | $m, s$    | (1, 1, 2, 3, 5, 7, 11)  |
|                        | $e$       | (1, 1, 2, 3, 4, 6, 8)   |
|                        | $p$       | (1, 2, 3, 5, 7, 11, 15) |
| $C_n, 3 \leq n \leq 7$ | $m, s$    | (1, 3, 3, 7, 8)         |
|                        | $e$       | (1, 2, 2, 4, 4)         |
|                        | $p$       | (3, 5, 7, 11, 15)       |
| $K_n, 1 \leq n \leq 7$ | $m, e, s$ | (1, 1, 1, 1, 1, 1, 1)   |
|                        | $p$       | (1, 2, 3, 5, 7, 11, 15) |

For paths and cycles in this range, the elementary expansion has the fewest terms. For complete graphs,

$$X_{K_n} = n!e_n,$$

so the elementary basis is especially natural. All three families have non-negative coefficients in the monomial, elementary, and Schur bases for these values of  $n$ . Negative power-sum coefficients occur as soon as  $n \geq 2$ .

4. **Solution.** Sage's `StarGraph(r)` has one center and  $r$  leaves, so it has  $r + 1$  vertices.

```
for n in range(1, 8):
    St = graphs.StarGraph(n-1)
    X = p(St.chromatic_symmetric_function(QQ))
    formula = sum((-1)^r * binomial(n-1,r)
                  * p[Partition([r+1] + [1]*(n-1-r))])
    for r in range(n)
    print(n, X, X == formula)
```

Every comparison returns `True`. Thus

$$X_{St_n} = \sum_{r=0}^{n-1} (-1)^r \binom{n-1}{r} p_{r+1, 1^{n-1-r}}.$$

This also follows directly from the edge-subset formula: choosing  $r$  of the  $n - 1$  star edges creates one component of size  $r + 1$  and  $n - 1 - r$  isolated vertices.

5. **Solution.**

```
from collections import defaultdict

for n in [7,8]:
    groups = defaultdict(list)
    trees = list(graphs.trees(n))
    for i,T in enumerate(trees, start=1):
        X = T.chromatic_symmetric_function(QQ)
        groups[X].append(i)
    collisions = [I for I in groups.values() if len(I) > 1]
    print(n, len(trees), collisions)
```

Sage generates one representative from each isomorphism class. It finds 11 trees for  $n = 7$  and 23 trees for  $n = 8$ . In both cases `collisions` is empty. Thus no two trees in either collection have the same chromatic symmetric function.

6. **Solution.** The code below computes the elementary expansion and groups trees by the requested coefficient.

```
trees8 = list(graphs.trees(8))
groups = defaultdict(list)

for i,T in enumerate(trees8, start=1):
    XE = e(T.chromatic_symmetric_function(QQ))
    c = XE[Partition([4,2,1,1])]
```

```

groups[c].append(i)
print(i, XE, "coefficient =", c)

print({c:I for c,I in groups.items() if len(I) > 1})

```

With trees numbered in Sage's iterator order, the repeated groups are

| coefficient | tree indices |
|-------------|--------------|
| 0           | 1, 16        |
| 3           | 4, 10        |
| 6           | 2, 7         |
| 10          | 3, 14        |
| 12          | 9, 22        |
| 13          | 8, 18        |

Therefore the coefficient of  $e_{4,2,1,1}$  does not distinguish all trees on eight vertices.

7. **Solution.** This program computes all five requested statistics.

```

for i,T in enumerate(trees8, start=1):
    X = T.chromatic_symmetric_function(QQ)
    XE, XP = e(X), p(X)
    deg = T.degree_sequence()
    leaves = deg.count(1)
    print(i,
          "degree sequence =", deg,
          "diameter =", T.diameter(),
          "leaves =", leaves,
          "e[7,1] =", XE[Partition([7,1])],
          "p[7,1] =", XP[Partition([7,1])])

```

The coefficient pair depends only on the number  $L$  of leaves:

| $L$            | 2 | 3  | 4  | 5  | 6  | 7  |
|----------------|---|----|----|----|----|----|
| $[e_{7,1}]X_T$ | 6 | 13 | 20 | 27 | 34 | 41 |
| $[p_{7,1}]X_T$ | 2 | 3  | 4  | 5  | 6  | 7  |

In fact, for a tree on  $n$  vertices,

$$[p_{n-1,1}]X_T = (-1)^{n-2}L, \quad [e_{n-1,1}]X_T = (n-1)L - n.$$

For the power coefficient, a contributing edge subset must isolate one leaf and contain all edges of the remaining  $(n-1)$ -vertex tree. Thus these coefficients recover the number of leaves, which is already visible in the degree sequence. They do not determine the diameter: different trees with the same number of leaves can have different diameters.

## Mini-Projects

- A. **Solution.** The following code produces a reproducible data record for every tree.

```
def basis_statistics(f):
    coeffs = f.coefficients()
    return {
        "terms": len(f.support()),
        "smallest coefficient": min(coeffs),
        "largest coefficient": max(coeffs),
        "all positive": all(c > 0 for c in coeffs)
    }

records = []
for i,T in enumerate(trees8, start=1):
    X = T.chromatic_symmetric_function(QQ)
    row = {"tree": i, "graph6": T.graph6_string()}
    for name,B in [("m",m),("e",e),("p",p),("s",s)]:
        row[name] = basis_statistics(B(X))
    records.append(row)
    print(row)
```

The total numbers of nonzero terms over all 23 trees are

| basis       | $m$   | $e$   | $p$   | $s$   |
|-------------|-------|-------|-------|-------|
| total terms | 384   | 377   | 416   | 401   |
| average     | 16.70 | 16.39 | 18.09 | 17.43 |

There is no basis that is sparsest for every tree. Counting ties, the elementary basis is sparsest for 15 trees, the monomial and power bases for 7 each, and the Schur basis for 3.

All 23 monomial expansions have positive coefficients, as follows directly from the coloring definition. Only one of these trees (the path) has all positive elementary coefficients in this experiment, and 9 have all positive Schur coefficients. None has all positive power-sum coefficients. The coefficients also encode graph data: for example, Exercise 7 shows how to recover the number of leaves.

- B. **Solution.** Sage partitions can be sorted directly in lexicographic order.

```
for i,T in enumerate(trees8, start=1):
    X = T.chromatic_symmetric_function(QQ)
    print("tree", i)
    for name,B in [("m",m),("e",e),("p",p),("s",s)]:
        support = sorted(B(X).support())
        print(name, "smallest =", support[0],
              "largest =", support[-1])
```

In the monomial basis, the smallest partition is always  $1^8$ . The first part of the largest partition is the independence number  $\alpha(T)$ : a color class is an independent set, and lexicographic maximization first makes the largest color class as large as possible. Thus the largest monomial index contains genuine information about the tree. In the power basis, connectedness forces a nonzero  $p_8$  term, so the largest index is always (8) and is not useful for distinguishing these trees by itself.

To study the effect of adding a leaf, attach it at every possible vertex and recompute the dictionaries of coefficients:

```
def add_leaf(T, v):
    U = T.copy()
    new_vertex = max(U.vertices(sort=False)) + 1
    U.add_edge(v, new_vertex)
    return U

for i,T in enumerate(trees8, start=1):
    for v in T.vertices(sort=False):
        U = add_leaf(T, v)
        XU = U.chromatic_symmetric_function(QQ)
        print("tree", i, "attachment vertex", v)
        for name,B in [("m",m),("e",e),("p",p),("s",s)]:
            print(name, B(XU).monomial_coefficients())
```

There is no single coefficient change independent of the attachment vertex. Attaching at different vertices can produce nonisomorphic trees and different expansions. The code makes that dependence explicit and repeats the experiment in all four bases.